Week 1, Gradient descent

CAPP 30254, Machine Learning for Public Policy

Least squares

Recall the closed-form expression of the least squares weight vector:

$$\mathbf{w}_{LS} = (X^T X)^{-1} X^T \mathbf{y}$$

when the columns of X are linearly independent.

It is computationally costly to invert a matrix, so computing $\mathbf{w}_{LS}$ might be prohibitively expensive.

Still, we might want to use least squares to model our machine learning problem.

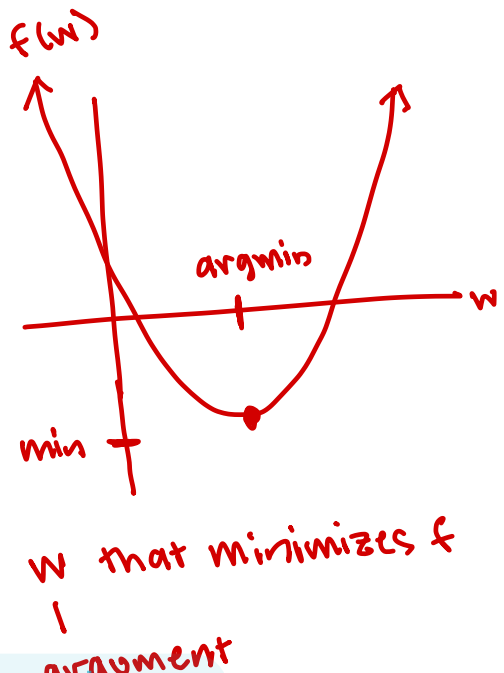
Optimization

We can cast least squares as an optimization problem by noticing that the least squares weight vector $\mathbf{w}_{LS}$ is the vector that minimizes the least squares loss function ℓ:

$$\mathbf{w}_{LS} = \textit{argmin}_{\mathbf{w}} \underbrace{\|\mathbf{y} - X\mathbf{w}\|_2^2}_{\text{same } \ell \text{ from before}}$$

$\hat{y}$

$$= \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$



Optimization

We can cast least squares as an optimization problem by noticing that the least squares weight vector $\mathbf{w}_{LS}$ is the vector that minimizes the least squares loss function ℓ:

$$\mathbf{w}_{LS} = \underset{\mathbf{w}}{argmin} \|\mathbf{y} - X\mathbf{w}\|_2^2$$

How do we solve for $\mathbf{w}_{LS}$?

Gradient descent is an algorithm used in optimization problems to find the minimum of a function by taking gradients.

The gradient

The *gradient* of a function $f(\mathbf{w})$ with respect to $\mathbf{w}$ is:

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \vdots \\ \frac{\partial f}{\partial w_n} \end{bmatrix}$$

derivative of f in each direction w_i

The gradient is a generalization of the derivative to higher dimensions. Geometrically, the gradient is a vector that points in the direction of steepest ascent.

The gradient

For example, let $\mathbf{w} \in \mathbb{R}^2$ and $f(\mathbf{w})$ be the function:

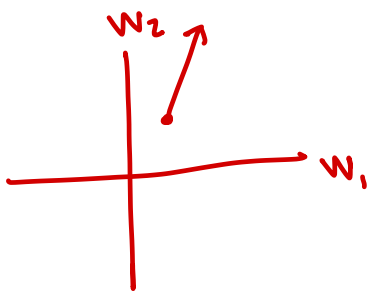
$$f(w_1, w_2) = w_1^2 + w_2^2$$

The gradient of this function with respect to w_1 and w_2 is:

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix} \leftarrow 1 \quad \leftarrow 2$$

$\underline{w} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\nabla_{\underline{w}} f(\underline{w}) = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$



Convex functions

A function is *convex* if:

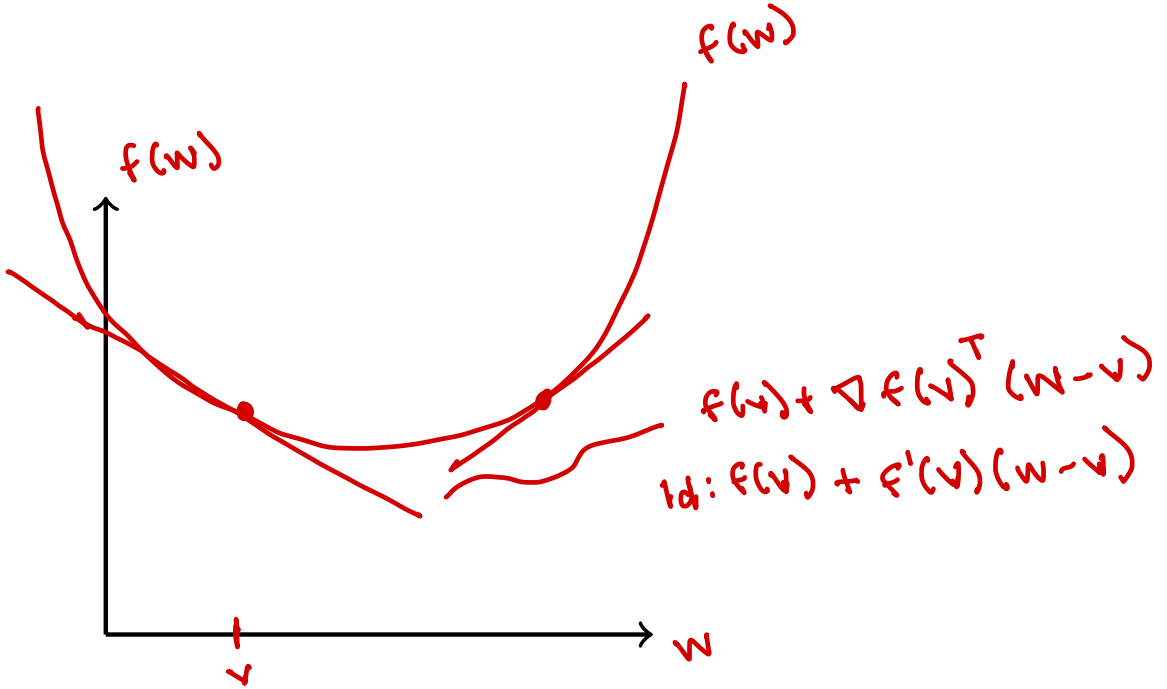
$$f(\mathbf{w}) \geq f(\mathbf{v}) + \nabla f(\mathbf{v})^T(\mathbf{w} - \mathbf{v})$$

"bowl shaped"

for all $\mathbf{v}$ and $\mathbf{w}$.

f is above tangent line at all points

Example in $\mathbb{R}$:



Not convex

Here's a function that is not convex in $\mathbb{R}$:

Gradient descent

Finds the argmin of a function

Given a loss function ℓ, the gradient descent algorithm iteratively updates $\mathbf{w}$ in the direction of the negative gradient in order to minimize the loss.

* The update rule is:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_w \ell(\mathbf{w}^{(t)})$$

where:

- $\mathbf{w}^{(t)}$ is the weight vector at iteration t
- η is the learning rate / step size

↑ eta

Gradient descent

The algorithm is:

- Pick an initial guess $\mathbf{w}^{(0)}$
- For each iteration $t = 1, 2, \ldots$ update the weight vector using the rule:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_w \ell(\mathbf{w}^{(t)})$$

converged

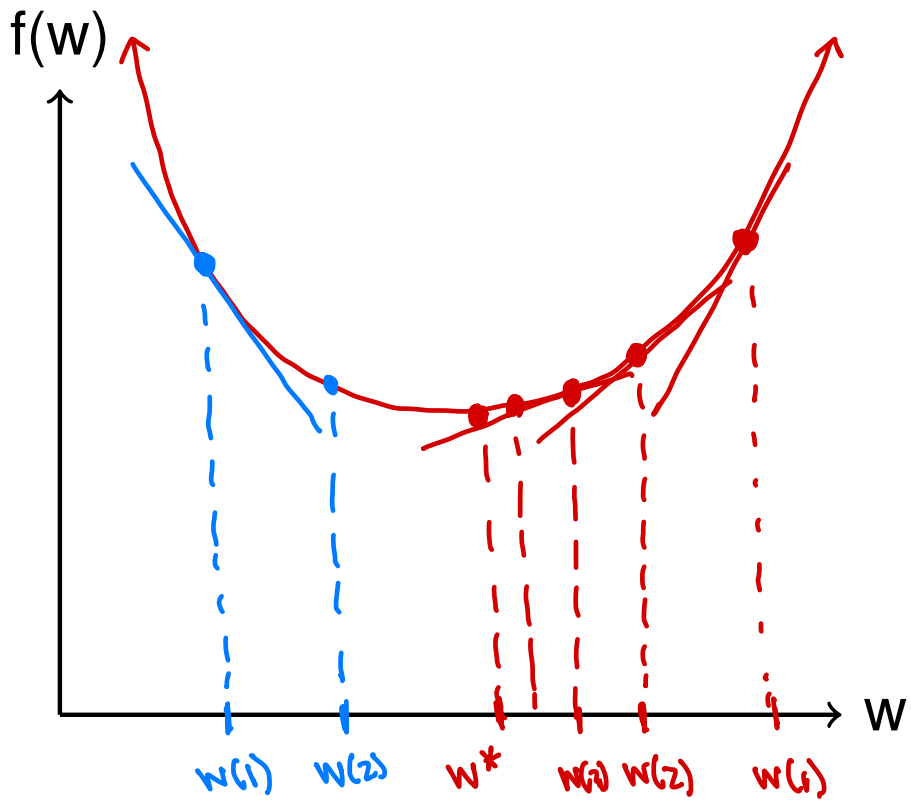
- Repeat until $\left\| w^{(t+1)} - w^{(t)} \right\|_2 < \epsilon$ for some small number ϵ or for some specified number of iterations

$\nabla > 0$

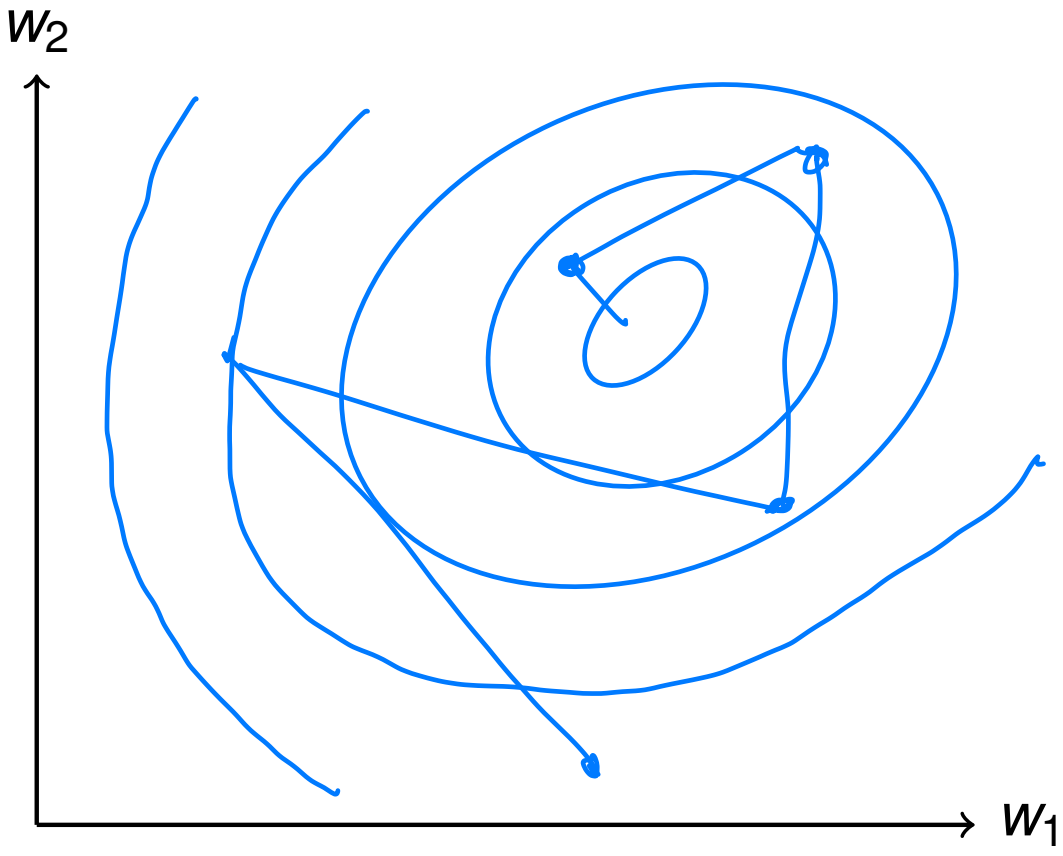
want to move left

Example in $\mathbb{R}$

$w \in \mathbb{R}$



Example in $\mathbb{R}^2$



What about η?

For least squares, gradient descent converges when:

$$\eta < \frac{1}{\sigma^2_{max}(X)}$$

Otherwise, it depends on ℓ.

A good choice for η in least squares-like problems is:

$$\eta \approx \frac{1}{\sigma^2_{max}(X)}$$

What about $\mathbf{w}^{(1)}$?

If ℓ is convex, gradient descent converges to the global minimum for any choice of $\mathbf{w}^{(1)}$.

If ℓ is not convex, gradient descent weight vector will depend on the initial choice of $\mathbf{w}^{(1)}$.

Stochastic gradient descent

GD: uses every training sample in every step

SGD: one training sample per step

- Pick an initial guess $\mathbf{w}^{(1)}$
- For each iteration $t = 1, 2, \ldots$ pick an $i \in \{1, \ldots, n\}$ and update the weight vector using the rule:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_w \ell_i(\mathbf{w}^{(t)})$$

Each iteration is fast but noisy

where ℓ_i is the component of the loss coming from training sample i

- Repeat until $\left\|\mathbf{w}^{(t+1)} - \mathbf{w}^{(t)}\right\|_2 < \epsilon$ for some small number ϵ or for some specified number of iterations

$\ell_i(\underline{w}) = (y_i - \underline{x}_i \underline{w})^2$ error from $\underline{x}_i$

Each iteration is faster,

Example in $\mathbb{R}^2$

How do we choose i?

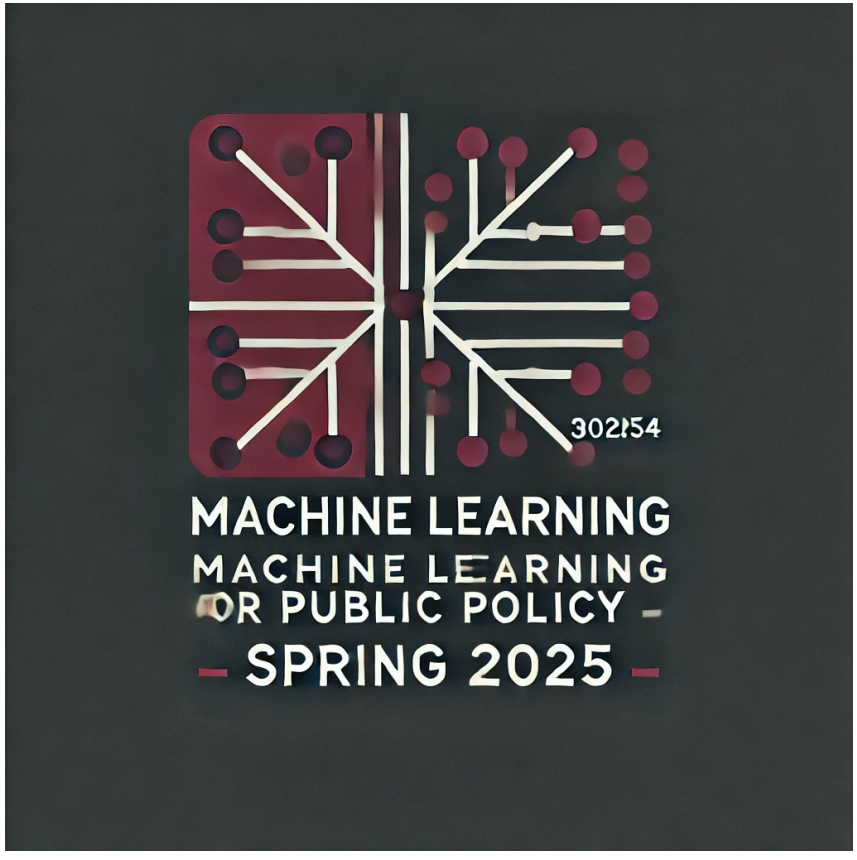
There are a number of ways to choose i for SGD.

- Choose i randomly from a uniform distribution
- In iteration t, let $i = t \bmod n$ 1, 2, 3, 1, 2, 3, 1, 2, 3
- * Use random permutations: Permute $\{1, \ldots, n\}$ and reshuffle every n rounds (epochs)

Ex. $n = 3$

i: 3, 2, 1, 2, 1, 3, 1, 3, 2, 3, 2, 1

epoch 1 epoch 2 epoch 3



[1]

[1] Generated by GPT-4, DALL-E 3 after the prompt: “Hi, could you please make a minimalistic logo for a machine learning class...” then “Could you make it less symmetrical?”